NutriTrack API Documentation

Overview

NutriTrack is a RESTful API for food nutrition tracking and dietary analysis, powered by the USDA SR Legacy dataset (7,793 foods). It provides endpoints for food search, meal logging, nutritional analytics, and user profile management.

- **OpenAPI Version**: 3.1.0
- **API Version**: 0.1.0

Base URL

Environment	URL
Local Development	`http://127.0.0.1:8000`
Production (PythonAnywhere)	`https://lichenxi.pythonanywhere.com`

Interactive documentation is available at:

- **Swagger UI**: `{BASE_URL}/docs`
- **ReDoc**: `{BASE_URL}/redoc`

Authentication

The API uses **JWT (JSON Web Token)** Bearer authentication via the OAuth2 password flow.

Obtaining a Token

1. Register a user account via `POST /api/auth/register`
2. Login via `POST /api/auth/login` to receive an access token
3. Include the token in the `Authorization` header for protected endpoints:

```
Authorization: Bearer <access_token>
```

Tokens expire after **30 minutes** by default.

Protected vs Public Endpoints

Access Level	Endpoints
Public	Auth (register, login), Foods (list, search, detail, categories)
Protected	Meals (CRUD), Analytics (daily, weekly, balance), User Profile

---

Error Responses

All error responses follow a consistent format:

Standard Errors

```
{
  "detail": "Error message describing what went wrong"
}
```

Status Code	Meaning
400	Bad Request — Invalid input (e.g., duplicate username)
401	Unauthorized — Missing or invalid JWT token
404	Not Found — Resource does not exist
422	Validation Error — Request body/params failed validation

Validation Error Format (422)

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": ["body", "field_name"],
      "msg": "Human-readable error message",
      "input": "the invalid value"
    }
  ]
}
```

Endpoints

0. Root

GET `/`

Health check endpoint — confirms the API is running.

Example Request:

```
curl https://lichenxi.pythonanywhere.com/
```

Response `200 OK`:

```
{
  "message": "NutriTrack API is running"
}
```

---

1. Auth

POST `/api/auth/register`

Register a new user account.

Request Body (`application/json`):

Field	Type	Required	Constraints	Description
username	string	Yes	3–50 chars, `^[a-zA-Z0-9_]+$`	Unique username
email	string (email)	Yes	Valid email format	Unique email address
password	string	Yes	6–128 chars	Account password

Example Request:

```
curl -X POST https://lichenxi.pythonanywhere.com/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username": "testuser", "email": "test@example.com", "password": "test1234"}'
```

Response `201 Created`:

```
{
  "id": 1,
  "username": "testuser",
  "email": "test@example.com"
}
```

Errors: `400` (username/email already registered), `422` (validation failed)

POST `/api/auth/login`

Login and receive a JWT token.

Request Body (`application/x-www-form-urlencoded`):

Field	Type	Required	Description
username	string	Yes	Registered username
password	string	Yes	Account password

Example Request:

```
curl -X POST https://lichenxi.pythonanywhere.com/api/auth/login \
  -d "username=testuser&password=test1234"
```

Response `200 OK`:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...",
  "token_type": "bearer"
}
```

Errors: `401` (incorrect credentials), `422` (validation failed)

---

2. Foods

GET `/api/foods/categories`

List all food categories.

Example Request:

```
curl https://lichenxi.pythonanywhere.com/api/foods/categories
```

Response `200 OK`:

```
[
  "American Indian/Alaska Native Foods",
  "Baby Foods",
  "Baked Products",
  "Beef Products",
  "Beverages",
  "Dairy and Egg Products",
  "Fruits and Fruit Juices"
]
```

GET `/api/foods/`

List foods with pagination and optional category filtering.

Query Parameters:

Parameter	Type	Default	Constraints	Description
page	integer	1	≥ 1	Page number
per_page	integer	20	1–100	Items per page
category	string	null	Optional	Filter by category name

Example Request:

```
curl "https://lichenxi.pythonanywhere.com/api/foods/?
page=1&per_page=3&category=Fruits%20and%20Fruit%20Juices"
```

Response `200 OK`:

```
{
  "items": [
```

```
    {
      "id": 4567,
      "fdc_id": 171688,
      "name": "Apples, raw, fuji, with skin",
      "category": "Fruits and Fruit Juices",
      "calories": 63.0,
      "protein": 0.2,
      "fat": 0.13,
      "carbs": 15.34,
      "fiber": 2.1
    }
  ],
  "total": 328,
  "page": 1,
  "per_page": 3
}
```

GET `/api/foods/search`

Search foods by name with relevance ranking and optional category filter.

Results are ranked by relevance:

1. Names **starting with** the query
2. Names **containing** the query (at word boundary)
3. Names **containing** the query (anywhere)

Query Parameters:

Parameter	Type	Required	Constraints	Description
q	string	Yes	1–100 chars	Search keyword
category	string	No	—	Filter by category
page	integer	No	≥ 1, default 1	Page number
per_page	integer	No	1–100, default 20	Items per page

Example Request:

```
curl "https://lichenxi.pythonanywhere.com/api/foods/search?q=chicken+breast&per_page=3"
```

Response `200 OK`:

```
{
  "items": [
    {
      "id": 1234,
      "fdc_id": 171077,
      "name": "Chicken breast, raw",
      "category": "Poultry Products",
      "calories": 120.0,
      "protein": 22.5,
```

```
      "fat": 2.62,
      "carbs": 0.0,
      "fiber": 0.0
    }
  ],
  "total": 42,
  "page": 1,
  "per_page": 3
}
```

Errors: `422` (empty query)

GET `/api/foods/{food_id}`

Get detailed nutritional information for a specific food.

Path Parameters:

Parameter	Type	Description
food_id	integer	Food ID

Example Request:

```
curl https://lichenxi.pythonanywhere.com/api/foods/1234
```

Response `200 OK`:

```
{
  "id": 1234,
  "fdc_id": 171077,
  "name": "Chicken breast, raw",
  "category": "Poultry Products",
  "calories": 120.0,
  "protein": 22.5,
  "fat": 2.62,
  "carbs": 0.0,
  "fiber": 0.0
}
```

Errors: `404` (food not found)

---

3. Meals *(requires authentication)*

POST `/api/meals/`

Log a new meal with food items.

Request Body (`application/json`):

Field	Type	Required	Constraints	Description
date	string (date)	Yes	YYYY-MM-DD, not in future	Meal date
meal_type	string	Yes	`breakfast` / `lunch` / `dinner` / `snack`	Type of meal
notes	string	No	—	Optional notes
items	array	Yes	≥ 1 item	Food items in the meal
items[].food_id	integer	Yes	—	ID of the food
items[].quantity	number	Yes	0–10000 (grams)	Quantity in grams

Example Request:

```
curl -X POST https://lichenxi.pythonanywhere.com/api/meals/ \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "date": "2026-04-13",
    "meal_type": "lunch",
    "notes": "Post-workout meal",
    "items": [
      {"food_id": 1234, "quantity": 200},
      {"food_id": 4567, "quantity": 150}
    ]
  }'
```

Response `201 Created`:

```
{
  "id": 1,
  "user_id": 1,
  "date": "2026-04-13",
  "meal_type": "lunch",
  "notes": "Post-workout meal",
  "items": [
    {
      "id": 1,
      "food_id": 1234,
      "quantity": 200.0,
      "food": {
        "id": 1234,
        "fdc_id": 171077,
        "name": "Chicken breast, raw",
        "category": "Poultry Products",
        "calories": 120.0,
        "protein": 22.5,
        "fat": 2.62,
        "carbs": 0.0,
        "fiber": 0.0
```

```
      }
    }
  ]
}
```

Errors: `401` (unauthorized), `404` (food not found), `422` (validation failed)

GET `/api/meals/`

List the current user's meal records.

Query Parameters:

Parameter	Type	Default	Constraints	Description
page	integer	1	≥ 1	Page number
per_page	integer	20	1–100	Items per page

Example Request:

```
curl -H "Authorization: Bearer $TOKEN" \
  "https://lichenxi.pythonanywhere.com/api/meals/?page=1&per_page=5"
```

Response `200 OK`:

```
{
  "items": [ /* array of MealResponse objects */ ],
  "total": 15
}
```

GET `/api/meals/{meal_id}`

Get details of a specific meal. Users can only access their own meals.

Response `200 OK`: `MealResponse` object

Errors: `401` (unauthorized), `404` (not found or not owned)

PUT `/api/meals/{meal_id}`

Update a meal. All fields are optional — only provided fields are updated.

Request Body (`application/json`):

Field	Type	Required	Description
date	string (date)	No	New date (not in future)
meal_type	string	No	`breakfast` / `lunch` / `dinner` / `snack`
notes	string	No	Updated notes

Field	Type	Required	Description
items	array	No	Replaces all items if provided (≥ 1)

Response `200 OK`: Updated `MealResponse` object

DELETE `/api/meals/{meal_id}`

Delete a meal record.

Response `204 No Content`

Errors: `401` (unauthorized), `404` (not found or not owned)

---

4. Analytics *(requires authentication)*

GET `/api/analytics/daily`

Get nutritional summary for a specific date.

Query Parameters:

Parameter	Type	Required	Description
date	string (date)	Yes	Date in YYYY-MM-DD format

Example Request:

```
curl -H "Authorization: Bearer $TOKEN" \
  "https://lichenxi.pythonanywhere.com/api/analytics/daily?date=2026-04-13"
```

Response `200 OK`:

```
{
  "date": "2026-04-13",
  "total": {
    "calories": 1850.5,
    "protein": 95.2,
    "fat": 62.3,
    "carbs": 220.8,
    "fiber": 18.5
  },
  "meal_count": 3
}
```

GET `/api/analytics/weekly`

Get 7-day nutritional trend starting from a given date.

Query Parameters:

Parameter	Type	Required	Description
start	string (date)	Yes	Start date (YYYY-MM-DD), shows 7 days

Response `200 OK`:

```
{
  "start_date": "2026-04-07",
  "end_date": "2026-04-13",
  "daily": [
    {
      "date": "2026-04-07",
      "total": {"calories": 0, "protein": 0, "fat": 0, "carbs": 0, "fiber": 0},
      "meal_count": 0
    },
    {
      "date": "2026-04-08",
      "total": {"calories": 2100.3, "protein": 110.5, "fat": 70.2, "carbs": 250.1, "fiber": 22.0},
      "meal_count": 3
    }
  ],
  "average": {
    "calories": 1350.2,
    "protein": 72.1,
    "fat": 45.6,
    "carbs": 162.3,
    "fiber": 14.2
  }
}
```

GET `/api/analytics/balance`

Compare daily intake against recommended nutritional values. If the user has set profile information (age, gender, height, weight, activity level), the recommended values are personalized using the **Mifflin-St Jeor equation** for BMR calculation.

Query Parameters:

Parameter	Type	Required	Description
date	string (date)	Yes	Date in YYYY-MM-DD format

Response `200 OK`:

```
{
  "date": "2026-04-13",
  "items": [
    {
      "nutrient": "calories",
      "actual": 1850.5,
      "recommended": 2200.0,
      "percentage": 84.1,
      "status": "ok"
```

```
    },
    {
      "nutrient": "protein",
      "actual": 95.2,
      "recommended": 55.0,
      "percentage": 173.1,
      "status": "high"
    },
    {
      "nutrient": "fat",
      "actual": 62.3,
      "recommended": 73.3,
      "percentage": 85.0,
      "status": "ok"
    },
    {
      "nutrient": "carbs",
      "actual": 220.8,
      "recommended": 275.0,
      "percentage": 80.3,
      "status": "ok"
    },
    {
      "nutrient": "fiber",
      "actual": 18.5,
      "recommended": 25.0,
      "percentage": 74.0,
      "status": "low"
    }
  ]
}
```

Balance Status Values:

Status	Meaning
`low`	Actual < 80% of recommended
`ok`	Actual is 80%–120% of recommended
`high`	Actual > 120% of recommended

---

5. User Profile *(requires authentication)*

GET `/api/users/profile`

Get the current user's profile.

Response `200 OK`:

```
{
  "id": 1,
  "username": "testuser",
  "email": "test@example.com",
  "height": 175.0,
  "weight": 70.0,
  "age": 25,
  "gender": "male",
  "activity_level": "moderate",
  "warnings": []
}
```

PUT `/api/users/profile`

Update user profile. All fields are optional — only provided fields are updated. The API uses **two-tier validation**: hard rejection for impossible values, soft warnings for extreme-but-possible values.

Request Body (`application/json`):

Field	Type	Constraints	Description
height	number	0–300 cm	Height in centimetres
weight	number	0–500 kg	Weight in kilograms
age	integer	1–130	Age in years
gender	string	`male` / `female`	Gender
activity_level	string	`sedentary` / `light` / `moderate` / `active` / `very_active`	Activity level

Response `200 OK`:

```
{
  "id": 1,
  "username": "testuser",
  "email": "test@example.com",
  "height": 195.0,
  "weight": 130.0,
  "age": 85,
  "gender": "male",
  "activity_level": "active",
  "warnings": [
    "Weight 130.0 kg is unusually high. Please verify.",
    "Age 85 with activity level 'active' is unusual. Please verify."
  ]
}
```

Data Models

FoodResponse

Field	Type	Description
id	integer	Internal food ID
fdc_id	integer / null	USDA FoodData Central ID
name	string	Food name
category	string	Food category
calories	number	Calories per 100g (kcal)
protein	number	Protein per 100g (g)
fat	number	Fat per 100g (g)
carbs	number	Carbohydrates per 100g (g)
fiber	number	Fiber per 100g (g)

NutrientSummary

Field	Type	Description
calories	number	Total calories (kcal)
protein	number	Total protein (g)
fat	number	Total fat (g)
carbs	number	Total carbohydrates (g)
fiber	number	Total fiber (g)

NutrientBalance

Field	Type	Description
nutrient	string	Nutrient name
actual	number	Actual intake
recommended	number	Recommended daily intake
percentage	number	`actual / recommended * 100`
status	string	`low` / `ok` / `high`